

Weeks 5-6: Recursion and Array Sorting

Gruppenummer: 27

Gruppemedlem:	Studienummer:
August Due Iversen	202505524
Frederik Buskbjerg Reichle	202506410
Otto German Jørgensen	202500600

Exercise 1:

Code is implemented and tested in exercise1/test.cpp

Function	Worst case	Argument
searchAlgorithm()	$O(N)$	In the worst case scenario the function will iterate through the array and find the last element or the element is not there
maxRecursive()	$O(N)$	This function will need to compare all the elements in the array either way
minRecursive()	$O(N)$	This function will also need to compare all the elements in the array either way

Exercise 2:

Code is implemented and tested in exercise2/main.cpp. Our code is explained within the code file.

Exercise 3:

Code is implemented and tested in exercise3/main.cpp. Our code is explained within the code file.

Exercise 4:

Code is implemented and tested in exercise4/main.cpp. Our code is explained within the code file.

Exercise 5:

Code is implemented in exercise5/simple_list.tpp and tested in exercise5/main.cpp

The Recursion Recipe

Everything we have done so far, in four steps.

1. State the problem in terms of its size or complexity.
2. Find the **base case**, and show it can be solved without recursion.
3. Find the **recursive case**: assume the smaller problem is solved, and use it.
4. Check that the recursive case makes **progress** and reaches the base case.

Analyse, design, *then* code. You will be asked to write these four down.

1. State the problem

We want to find the index of an object (x) in the linked list ADT and return that index.

2. Find the base case

The base case would either be if x is equal to the data contained in the current node, or if the current node is the tail of the list (in which case -1 should be returned because the object isn't contained in the list).

3. Find the recursive case

The recursive case would be if x doesn't equal the current node while the tail hasn't yet been reached. In this case we should move on to the next node, calling the function recursively once again.

4. Check progress

As we move on to each next node, we will eventually either find a node containing x or reach the end of the list (the two base cases), no matter what.

Exercise 6:

The code is implemented and tested in exercise7 in the zip folder.

For this exercise we've implemented the algorithm selection sort as a template and tested it with integers and strings. As for the complexity of the algorithm, the worst- and best-case scenario is the same since both our for-loops always run the entire length with no break or return option in case it's already sorted. Therefore:

Best case: $O(N^2)$

Worst case: $O(N^2)$

Here it is worth noting that the inner loop gets progressively smaller but not fast enough to change the Big O.

Exercise 7:

In this exercise, the pseudocode from Wikipedia, which was referenced in the description, has been rewritten to fit in C++ with vectors.

```
std::vector<int> counting_sort(std::vector<int> A, int k)
{
    std::vector<int> count(k + 1, 0);
    std::vector<int> output(A.size());

    for (int i = 0; i < A.size(); i++)
    {
        int j = A[i];
        count[j] = count[j] + 1;
    }

    for (int i = 1; i <= k; i++)
    {
        count[i] = count[i] + count[i - 1];
    }

    for (int i = A.size() - 1; i >= 0; i--)
    {
        int j = A[i];
```

```

        count[j] = count[j] - 1;
        output[count[j]] = A[i];
    }
    return output;
}

```

(See counting_sort.cpp in the added files for test of function)

N can be defined as the size of the array, so in the exercise it's

$$N = k$$

The time complexity is $O(N)$ because it uses for-loops through the whole size of the array N and there are no nested loops.

The space complexity is $O(N)$ because two different vectors are created inside the function; **count** and **output**. **count** allocates $N+1$ space and **output** N space. The math for this will be

$$2N + 1$$

And the final big O, including only the dominant part for space complexity is therefore

$$O(N)$$

Exercise 8:

a)

Code is implemented in exercise8/quickSort.h and tested in exercise8/test.cpp

b)

Code of time measurement is implemented in exercise8/main.cpp

The chosen input sizes are:

```

vector<int> sizes = {2000, 4000, 6000, 8000, 10000, 12000, 14000, 16000, 18000, 20000};

```

We want all the test arrays to be of a relatively large size so that we can eliminate big fluctuations between runs using the law of large numbers. Also to get a good amount of samples we test 10 different sizes, of which every sample is equal to the latter incremented by the very first sample.

Initial run (useInsertion = 16):

```

Time taken for vector of size 2000: 250 microseconds
Time taken for vector of size 4000: 534 microseconds
Time taken for vector of size 6000: 853 microseconds
Time taken for vector of size 8000: 1156 microseconds
Time taken for vector of size 10000: 1470 microseconds
Time taken for vector of size 12000: 1796 microseconds
Time taken for vector of size 14000: 2114 microseconds
Time taken for vector of size 16000: 2442 microseconds
Time taken for vector of size 18000: 2861 microseconds
Time taken for vector of size 20000: 3174 microseconds

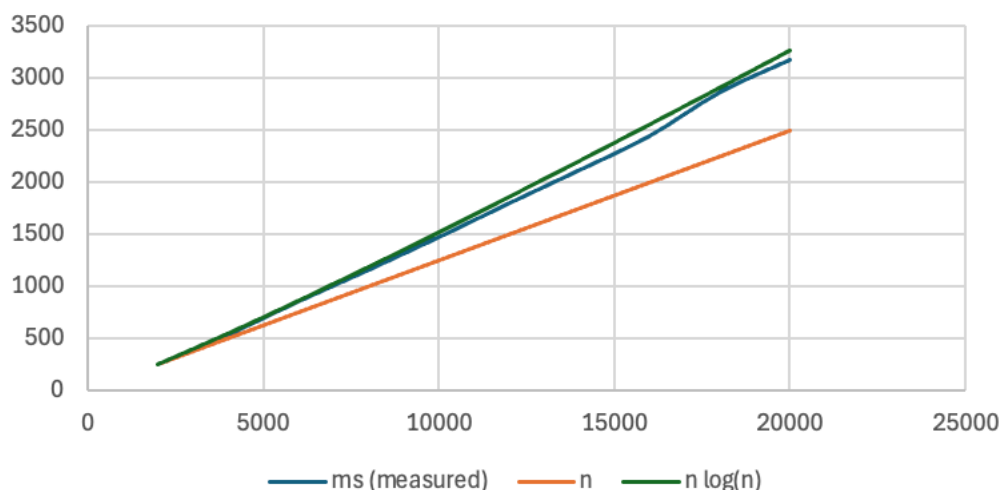
```

We know the time complexity of the kind of sort (introSort) is a guaranteed $O(n \log(n))$.

So let's compare our test data to graphs of n and $n \log(n)$:

n (samples)	ms (measured)	n	$n \log(n)$
2000	250	250	250
4000	534	500	545,596374
6000	853	750	858,402814
8000	1156	1000	1182,3855
10000	1470	1250	1514,67875
12000	1796	1500	1853,59475
14000	2114	1750	2198,01822
16000	2442	2000	2547,15649
18000	2861	2250	2900,41688
20000	3174	2500	3257,33938

Comparisson



As we can see, the actual measurements correlate best with $n \log(n)$, thus the time complexity is proven to be $O(n \log(n))$.

For experimentation with the useInsertion constant, we will test these values:

8, 16, 32, 64, 128

useInsertion = 8

```
Time taken for vector of size 2000: 248 microseconds
Time taken for vector of size 4000: 522 microseconds
Time taken for vector of size 6000: 832 microseconds
Time taken for vector of size 8000: 1133 microseconds
Time taken for vector of size 10000: 1435 microseconds
Time taken for vector of size 12000: 1747 microseconds
Time taken for vector of size 14000: 2094 microseconds
Time taken for vector of size 16000: 2386 microseconds
Time taken for vector of size 18000: 2767 microseconds
Time taken for vector of size 20000: 3290 microseconds
```

useInsertion = 16 (initial run)

```
Time taken for vector of size 2000: 250 microseconds
Time taken for vector of size 4000: 534 microseconds
Time taken for vector of size 6000: 853 microseconds
Time taken for vector of size 8000: 1156 microseconds
Time taken for vector of size 10000: 1470 microseconds
Time taken for vector of size 12000: 1796 microseconds
Time taken for vector of size 14000: 2114 microseconds
Time taken for vector of size 16000: 2442 microseconds
Time taken for vector of size 18000: 2861 microseconds
Time taken for vector of size 20000: 3174 microseconds
```

useInsertion = 32

```
Time taken for vector of size 2000: 277 microseconds
Time taken for vector of size 4000: 585 microseconds
Time taken for vector of size 6000: 921 microseconds
Time taken for vector of size 8000: 1248 microseconds
Time taken for vector of size 10000: 1591 microseconds
Time taken for vector of size 12000: 1949 microseconds
Time taken for vector of size 14000: 2294 microseconds
Time taken for vector of size 16000: 2660 microseconds
Time taken for vector of size 18000: 3131 microseconds
Time taken for vector of size 20000: 3517 microseconds
```

useInsertion = 64

```
Time taken for vector of size 2000: 362 microseconds
Time taken for vector of size 4000: 734 microseconds
Time taken for vector of size 6000: 1164 microseconds
Time taken for vector of size 8000: 1572 microseconds
Time taken for vector of size 10000: 1974 microseconds
Time taken for vector of size 12000: 2382 microseconds
Time taken for vector of size 14000: 3075 microseconds
Time taken for vector of size 16000: 3316 microseconds
Time taken for vector of size 18000: 3796 microseconds
Time taken for vector of size 20000: 4376 microseconds
```

useInsertion = 128

```
Time taken for vector of size 2000: 541 microseconds
Time taken for vector of size 4000: 1084 microseconds
Time taken for vector of size 6000: 1636 microseconds
Time taken for vector of size 8000: 2261 microseconds
Time taken for vector of size 10000: 2851 microseconds
Time taken for vector of size 12000: 3437 microseconds
Time taken for vector of size 14000: 4084 microseconds
Time taken for vector of size 16000: 4833 microseconds
Time taken for vector of size 18000: 5274 microseconds
Time taken for vector of size 20000: 5916 microseconds
```

As we can see, when increasing the useInsertion constant, the execution time increases. Though when decreasing it to 8, the actual measurements still align well with the original useInsertion = 16 measurements, decreasing the cut-off from 16 didn't actually improve the time complexity. And even if it could improve the execution time, it would still be a space-complexity trade-off, as recursion is more memory-expensive than a simple loop. In conclusion using 16 as the cut-off is optimal.

c)

Code of time measurement is implemented in exercise8/main.cpp

Measurements for quickSort:

```
Time taken for vector of size 2000: 251 microseconds
Time taken for vector of size 4000: 527 microseconds
Time taken for vector of size 6000: 884 microseconds
Time taken for vector of size 8000: 1153 microseconds
Time taken for vector of size 10000: 1450 microseconds
Time taken for vector of size 12000: 1791 microseconds
Time taken for vector of size 14000: 2115 microseconds
Time taken for vector of size 16000: 2564 microseconds
Time taken for vector of size 18000: 2797 microseconds
Time taken for vector of size 20000: 3253 microseconds
```

Measurements for stlsort:

```
Time taken for vector of size 2000: 50 microseconds
Time taken for vector of size 4000: 97 microseconds
Time taken for vector of size 6000: 145 microseconds
Time taken for vector of size 8000: 200 microseconds
Time taken for vector of size 10000: 254 microseconds
Time taken for vector of size 12000: 296 microseconds
Time taken for vector of size 14000: 305 microseconds
Time taken for vector of size 16000: 369 microseconds
Time taken for vector of size 18000: 442 microseconds
Time taken for vector of size 20000: 472 microseconds
```

Both algorithms are growing at roughly the same rate, though the stlsort measurements are performing better. This is because the std::sort algorithm is heavily optimized and has a much smaller constant factor, due to compiler optimizations.